



Gilson Ethernet Utility Sample Program Documentation

Revision History		
Rev.	Summary of Change	Date
01	Original	1/18/10

Gilson, Inc. World Headquarters

3000 Parmenter Street | P.O. Box 620027 | Middleton, WI 53562-0027, USA | Tel: 608-836-1551 | Fax: 608-831-4451

Gilson S.A.S. | 19, avenue des Entrepreneurs | BP 145, F-95400 VILLIERS LE BEL, France

www.gilson.com | sales@gilson.com | service@gilson.com | training@gilson.com

Table of Contents

1	Actions Performed by Sample Program	3
2	Listening for Beacon	3
3	Connecting to Instrument	4
4	Obtaining Lease or Admin Privileges	6
4.1	Obtaining Admin Privileges	6
4.2	Obtaining Lease	8
5	Controlling Instrument	10
5.1	Obtaining Instruction Set from Instrument	10
5.2	Sending Command to Instrument	13
6.	Command & Response List.....	15

Code Documentation

Listening for Beacon Sample Code 1	3
Connect to instrument handler Sample Code 2	5
Acquiring admin privileges Sample Code 3	7
Processing response from requesting admin privileges Sample Code 4	7
Obtaining Lease Sample Code 5	8
Obtaining instruction set Sample Code 6	10
Processing response to Get Instruction Set command Sample Code 7	11
Retrieving commands from the instruction set Sample Code 8	12
Constructing EthernetMessage fro the selected command Sample Code 9	14
Sending EthernetMessage to the instrument Sample Code 10	15

Screen Shots

List of Ethernet instruments available Screen Shot 1	4
Device context menu for instrument Screen Shot 2	4
Obtaining admin privileges Screen Shot 3	6
After obtaining admin privileges Screen Shot 4	7
Obtaining Lease Screen Shot 5	8
After obtaining lease Screen Shot 6	9
Obtaining instruction set Screen Shot 7	10
Commands from the instruction set Screen Shot 8	12
Editing parameters to the command Screen Shot 9	13
Command added to the Execution List Screen Shot 10	13
Command/Response list Screen Shot 11	15

1 Actions Performed by Sample Program

The sample program demonstrates use of the Gilson Ethernet Assembly. The sample program includes the following operations:

- Listens for Beacon data and identifies the instruments available on the Ethernet.
- Connects to the instrument using the Ethernet Assembly.
- Communicates with the instrument via System commands & Commands specific to instrument.
- Parses beacon data and receives responses from the instrument.

2 Listening for Beacon

A UDP listener is started on the beacon port. The UDP listener receives beacon packets from available instruments.

```
public SampleApplication()
{
    InitializeComponent();

    // Start the UDP Listener on BeaconPort. Call back method BeaconData
    //will be called for every incoming beacon found.
    udpListener = new EthernetUDPListener(
        new EthernetDataDelegate(BeaconData), GilsonPorts.BeaconPort);

    udpListener.Start();

    // Inititalize the DataGrid for Command Parameters.
    InitDataGrid();

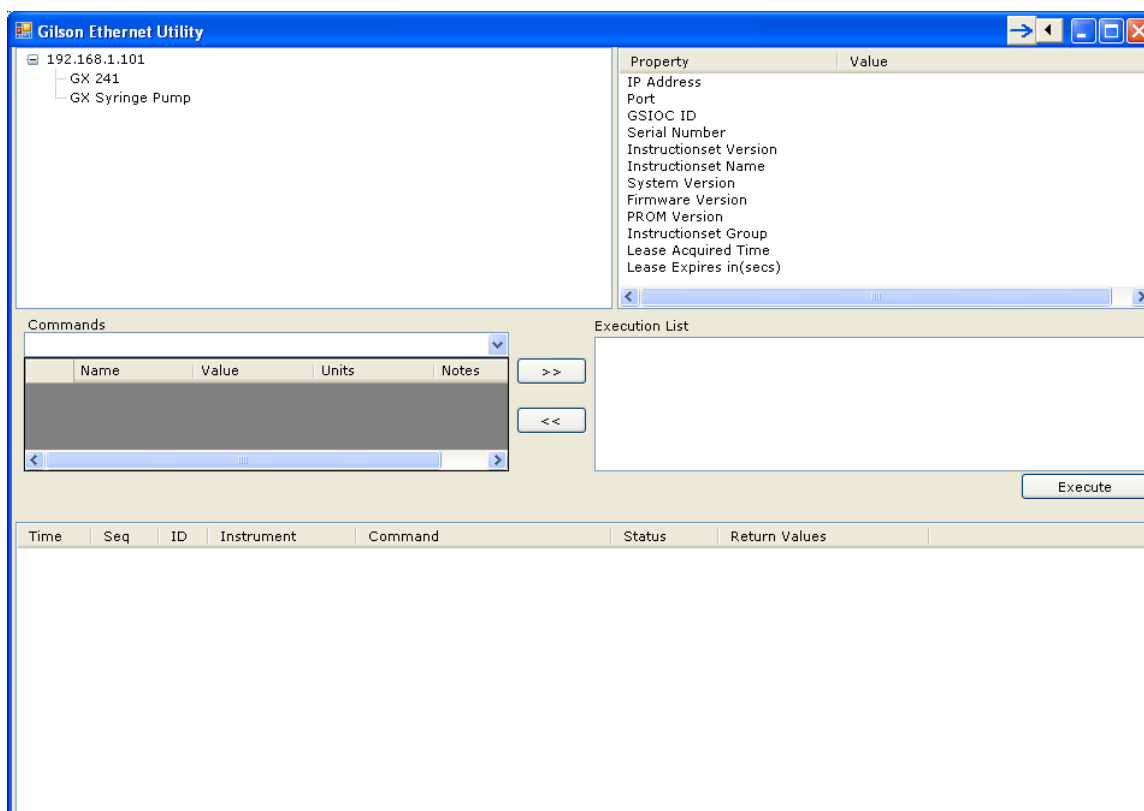
    // Start the Worker thread from the thread pool
    //to update the device status.
    ThreadPool.QueueUserWorkItem(
        new WaitCallback(ThreadVerifyConnectionStatus));
}
```

Listening for Beacon Sample Code 1

The worker thread *ThreadVerifyConnectionStatus()* is started to check if the instrument is active and available. The status of the instrument is checked every 10 seconds. The check is done by comparing the time of when the last beacon was received with the current time. If a beacon from the instrument was not received within the previous 10 seconds, then the instrument is considered to be disconnected from the Ethernet.

NOTE: Every Ethernet instrument sends out a beacon at a periodic rate (typically about every 2 seconds).

The following is a screen shot showing a GX-241 active and available on the Ethernet:

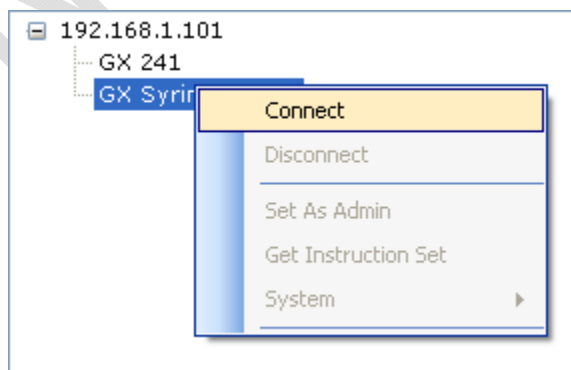


List of Ethernet instruments available Screen Shot 1

3 Connecting to Instrument

To connect to the instrument, select any instrument listed under the IP Address and then select the 'Connect' menu option from the device context menu.

The following is a screen shot of the device context menu:



Device context menu for instrument Screen Shot 2

Following is the code which handles the 'Connect' command:

```
private int ConnectToDevice(InstrumentDefinition instrumentDef)
{
    int port = -1;
    if (instrumentDef != null)
    {
        // Create instance of EthernetSend helper class which handles
        // the tcp communication to connect to device.
        EthernetSend ethernetSend = new EthernetSend();

        // Request a port to connect to from the device.
        port = ethernetSend.RequestPort(instrumentDef.IP, this.ToString());
        if (port > 0)
        {
            GilsonTcp tcpSocket = new GilsonTcp(
                System.Net.IPAddress.Parse(instrumentDef.IP), port);

            // Hook-up callbacks to be called when the Socket is connected,
            // Closed & when there is incoming data.
            tcpSocket.SocketClosed -= new SocketDelegate(SocketClosed);
            tcpSocket.SocketClosed += new SocketDelegate(SocketClosed);

            tcpSocket.SocketConnected -=
                new SocketDelegate(SocketConnected);

            tcpSocket.SocketConnected +=
                new SocketDelegate(SocketConnected);

            tcpSocket.AddListener(new EthernetDataDelegate(IncomingData));
            // Keep the new socket in the collection.
            socketList.Add(tcpSocket);
            treeInstrumentsAfterSelect(null, null);
        }
    }
    return port;
}
```

Connect to instrument handler Sample Code 2

This line, referenced from the Connect to instrument handler Sample Code 2, demonstrates sending a connection request to the instrument on the admin port and receiving back the subsequent port number to use for the TCP/IP connection.

```
port = ethernetSend.RequestPort(instrumentDef.IP, this.ToString());
```

After we receive the port information, we establish a TCP/IP connection to the instrument using the port assigned by the RequestPort method.

```
GilsonTcp tcpSocket = new GilsonTcp(
    System.Net.IPAddress.Parse(instrumentDef.IP), port);
```

After establishing the connection to the instrument, we need to obtain a lease or Admin privileges to control the instrument, though there are certain system commands which can be executed without obtaining the lease or admin privileges. Some of these system commands are available in the context shown in the figure: Device context menu for instrument Screen Shot 2.

The following code demonstrates how to add a delegate to our socket. This delegate is called for all the incoming data. Incoming data can be a response generated by a command or a status message from the instrument.

```
tcpSocket.AddListener(new EthernetDataDelegate(IncomingData));
```

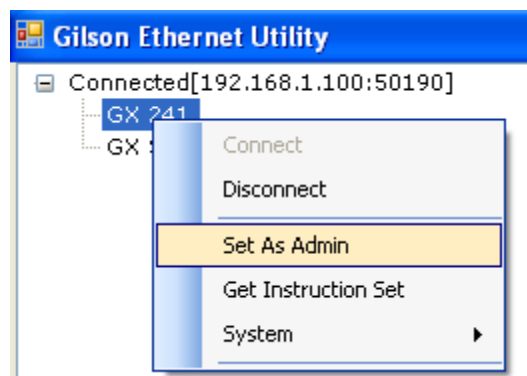
Following is an example of how to implementation of the delegate method assigned to handle the incoming message events. This code demonstrates how to determine the type of message received and process it accordingly:

```
private void IncomingData(EthernetReturnBase data)
{
    // If this form is created on a different
    // thread than the calling thread.
    if (this.InvokeRequired)
    {
        this.BeginInvoke(new EthernetDataDelegate(
            IncomingData), new object[] { data });
    }
    else
    {
        //Determine the type of response and handle accordingly
        if (data is EthernetResponseData)
        {
            ProcessResponse(data as EthernetResponseData);
        }
        else if (data is EthernetStatusData)
        {
            ProcessStatus(data as EthernetStatusData);
        }
    }
}
```

4 Obtaining Lease or Admin Privileges

4.1 Obtaining Admin Privileges

Admin privilege is obtained by selecting the menu option 'Set As Admin' from the device context menu.



Obtaining admin privileges Screen Shot 3

Following is the code which is executed when the 'Set As Admin' menu option is selected:

```
bool success = SendRequest(  
    socketList[currentInstrument.IP, currentInstrument.Port],  
    EthernetMessages.SetAsAdmin);
```

Acquiring admin privileges Sample Code 3

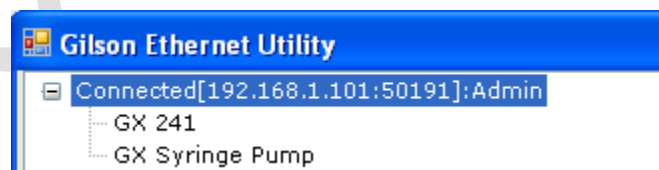
NOTE: Command used to acquire admin privileges is a System Command.

Following is the code for processing the response from the instrument.

```
private void ProcessResponse(EthernetResponseData responseData)  
{  
    // Get the tree node associated with the IP address from where  
    // response is sent.  
    ConnectionInformation connectionInfo = null;  
    TreeNode parentNode = treeInstruments.Nodes[responseData.IP];  
    if (parentNode != null)  
    {  
        connectionInfo = parentNode.Tag as ConnectionInformation;  
    }  
  
    if (ResponseCodeConverter.ConvertCode(  
        responseData.ResponseCode).Equals("Success"))  
    {  
        switch (responseData.CommandName)  
        {  
            case EthernetMessages.CommandAdmin:  
                //Set the connection to have admin rights  
                connectionInfo.IsAdmin = true;  
                parentNode.Text = string.Format("Connected[{0}:{1}]:{2}",  
                    connectionInfo.IP,  
                    connectionInfo.Port,  
                    connectionInfo.LeaseHolderString);  
                break;  
        }  
    }  
}
```

Processing response from requesting admin privileges Sample Code 4

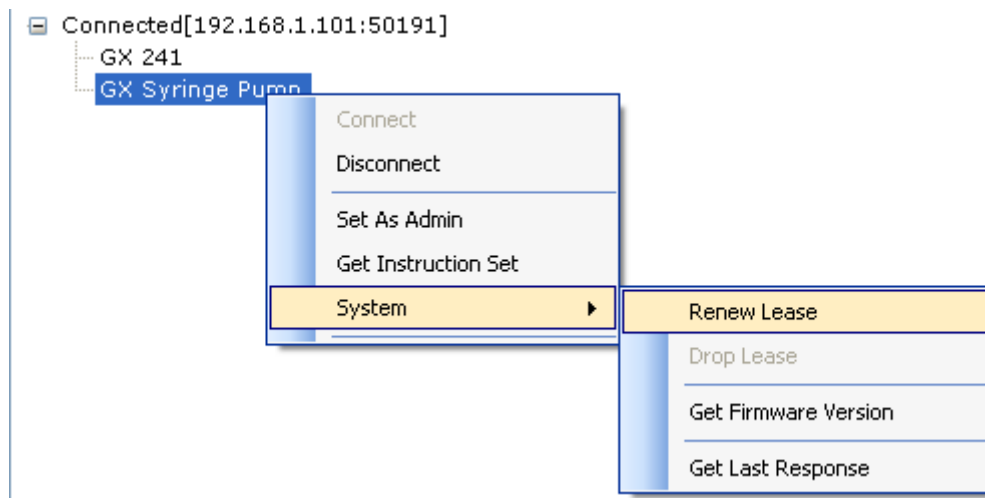
Following is the screen shot after obtaining the admin privileges.



After obtaining admin privileges Screen Shot 4

4.2 Obtaining Lease

Following is the screen shot of the menu option to obtain lease from the instrument.



Obtaining Lease Screen Shot 5

Following is the code which is run for the Renew Lease menu option.

```
bool success = SendRequest(  
    socketList[currentInstrument.IP,  
    currentInstrument.Port],  
    EthernetMessages.RenewLease);
```

Obtaining Lease Sample Code 5

NOTE: The command used to obtain lease is a System Command.

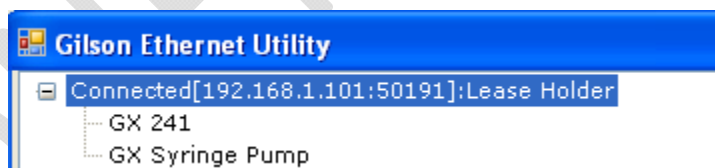
Following is the code for processing the response from the instrument.

```
private void ProcessResponse(EthernetResponseData responseData)
{
    // Get the tree node associated with the IP address from where
    // response is sent.
    ConnectionInformation connectionInfo = null;
    TreeNode parentNode = treeInstruments.Nodes[responseData.IP];
    if (parentNode != null)
    {
        connectionInfo = parentNode.Tag as ConnectionInformation;
    }

    if (ResponseCodeConverter.ConvertCode(
        responseData.ResponseCode).Equals("Success"))
    {
        switch (responseData.CommandName)
        {
            /** Other cases here ***/

            case EthernetMessages.CommandRenewLease:
                //Reset the lease time
                if (connectionInfo != null)
                {
                    connectionInfo.LeaseAcquiredTime =
                        DateTime.Now;
                }
                parentNode.Text =
                    string.Format("Connected[{0}:{1}]:{2}",
                        connectionInfo.IP,
                        connectionInfo.Port,
                        connectionInfo.LeaseHolderString);
                break;
        }
    }
}
```

Following is the screen shot after obtaining the lease

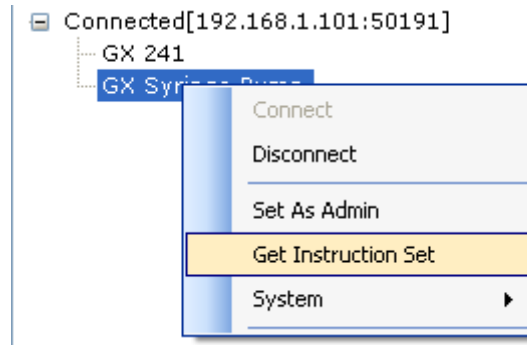


After obtaining lease Screen Shot 6

5 Controlling Instrument

5.1 Obtaining Instruction Set from Instrument

Following is the screen shot of the menu option to obtain the instruction set from the instrument.



Obtaining instruction set Screen Shot 7

Following is the code which is executed for this menu option:

```
bool success = SendRequest(  
    socketList[currentInstrument.IP,  
    currentInstrument.Port],  
    EthernetMessages.GetInstructionSet);
```

Obtaining instruction set Sample Code 6

NOTE: Command used to get the instruction set is a System command.

Following is the code processing the response from the instrument:

```
private void ProcessResponse(EthernetResponseData responseData)
{
    // Get the tree node associated with the IP address from where
    // response is sent.
    ConnectionInformation connectionInfo = null;
    TreeNode parentNode = treeInstruments.Nodes[responseData.IP];
    if (parentNode != null)
    {
        connectionInfo = parentNode.Tag as ConnectionInformation;
    }

    if (ResponseCodeConverter.ConvertCode(
        responseData.ResponseCode).Equals("Success"))
    {
        switch (responseData.CommandName)
        {
            /**/ Other cases here /**/

            case EthernetMessages.CommandGetInstructionSet:
                // Create the instance of class InstructionSetDeviceList
                //which represents the collection of instruction
                //sets for each device.
                InstructionSetDeviceList instructionSetList =
                    new InstructionSetDeviceList();

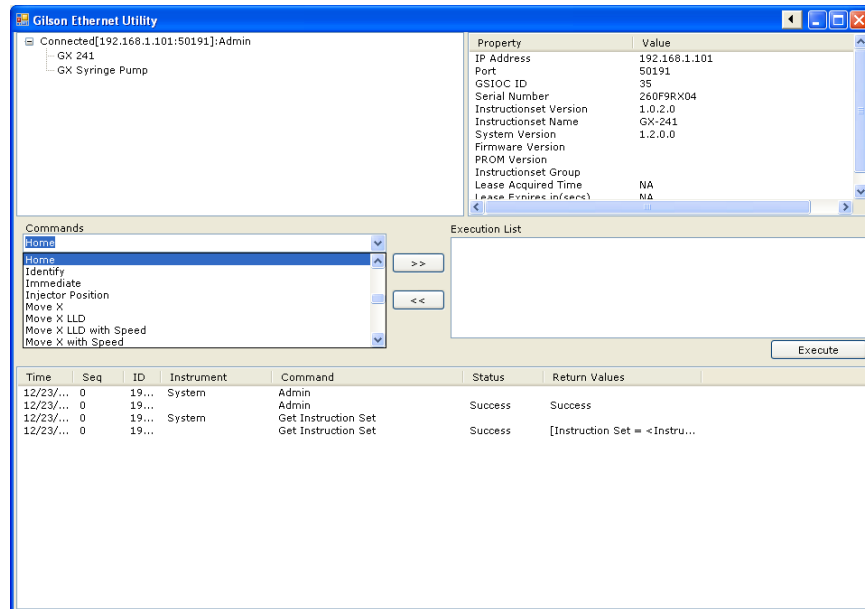
                // Call InstrumentSetDeviceList.ReadXML() to
                // deserialize instruction set XML and populate the
                // properties of InstrumentSetDeviceList class
                // object created above.
                instructionSetList.ReadXml("<Gilson>" +
                    responseData.EthernetReturnParameterList
                    ["Instruction Set"].ParamValue + "</Gilson>");

                // Loop through each of the devices and retrieve the
                // instruction sets for them
                foreach (TreeNode deviceNode in parentNode.Nodes)
                {
                    InstrumentDefinition instrumentDef =
                        deviceNode.Tag as InstrumentDefinition;

                    if (instrumentDef != null)
                    {
                        instrumentDef.InstructionSet = instructionSetList;
                    }
                }
                break;
        }
    }
}
```

Processing response to Get Instruction Set command Sample Code 7

Following is a screen shot after the instruction is processed and the commands are populated:



Commands from the instruction set Screen Shot 8

Following is the code populating the commands for the selected instrument:

```
private void UpdateCommandList(InstructionSetDeviceList instructionSet,
    string instructionSetName)
{
    if (instructionSet != null)
    {
        this.cmbCommands.Items.Clear();
        this.cmbCommands.Text = string.Empty;
        this.tblCmdParam.Clear();

        // Get the instruction set for the user selected device.
        InstructionSetDevice deviceInstructionSet =
            instructionSet[instructionSetName];

        if (deviceInstructionSet != null)
        {
            // Populate the commands from the instruction set.
            foreach (InstructionSetCommand command in
                deviceInstructionSet.Commands)
            {
                this.cmbCommands.Items.Add(command.Name);
            }
            cmbCommands.SelectedIndex = 0;
        }
    }
}
```

Retrieving commands from the instruction set Sample Code 8

5.2 Sending Command to Instrument

Parameters for a command are entered into the table directly under the combo box where the command is selected. In this screen shot, the command selected is the Home command and its only parameter is called 'Synchronized'. This parameter takes either True or False and can be changed by editing it within the value field.

Commands				
Home				
	Name	Value	Units	Notes
	Synchronized	False		

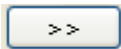
Execution List

>>

<<

Execute

Editing parameters to the command Screen Shot 9

To send these commands to the instrument add this command to the execution list by clicking on the  button.

Following is the screen shot after the command is added to the execution list.

Commands				
Home				
	Name	Value	Units	Notes
	Synchronized	False		

Execution List

☒ Home [GX-241 : 192.168.1.101:50192]

>>

<<

Execute

Command added to the Execution List Screen Shot 10

Following is the code building the Ethernet message to be sent to the instrument.

```
private EthernetMessage CreateEthernetMessage()
{
    EthernetMessage ethernetMsg = null;

    if (currentInstrument != null && socketList[
        currentInstrument.IP, currentInstrument.Port] != null)
    {
        // Create the EthernetMessage into which commands
        // will be inserted. EthernetMessage represents the
        // XML message sent to the ethernet instrument. Assign
        // the ethernet instruments IP Address and Port information.
        ethernetMsg = new EthernetMessage();
        ethernetMsg.IP = currentInstrument.IP;
        ethernetMsg.PortNumber = currentInstrument.Port;

        // Create the EthernetCommand object with Command Name,
        // Instruction Set Name(Device Name), GSIOC ID of the
        // Ethernet instrument, synchronized is set to false.
        EthernetCommand command = new EthernetCommand(
            cmbCommands.Text,
            currentInstrument.InstructionSetName,
            currentInstrument.UnitID,
            false);

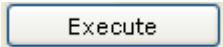
        command.Selected = true;

        // Populate the command with the command parameters.
        // Command parameter is represented by EthernetParameter class.
        foreach (DataRow dataRow in tblCmdParam.Rows)
        {
            if (dataRow[0].ToString().Equals("Synchronized"))
            {
                command.Synchronize =
                    dataRow[1].ToString().ToLower().Equals("true");
            }
            else
            {
                command.Parameters.Add(new EthernetParameter(
                    dataRow[0].ToString(), dataRow[1].ToString()));
            }
        }

        // Add the command object EthernetCommand created above to
        // message object EthernetMessage.
        ethernetMsg.Commands.Add(command);
    }

    return ethernetMsg;
}
```

Constructing EthernetMessage from the selected command Sample Code 9

To send a command, click  on the bottom of the Execution List.

Following is the code which handles sending the command to the instrument.

```
private void SendMessage(EthernetMessage message)
{
    GilsonTcp tcpSocket = null;
    if (currentInstrument != null &&
        socketList[currentInstrument.IP, currentInstrument.Port] != null)
    {
        try
        {
            // Get the socket connected to the IP:Port from the socket
            // list to which this message has to be sent.
            tcpSocket = socketList[message.IP, message.PortNumber];
            if (tcpSocket != null)
            {
                tcpSocket.TcpSend(message);
            }
        }
    }
}
```

Sending EthernetMessage to the instrument Sample Code 10

6. Command & Response List

Following is the screen shot listing the commands & responses to/from the instrument. Every command sent and the response generated by that command is added to the list.

Time	Seq	ID	Instrument	Command	Status	Return Values
12/23/...	0	19...	System	Renew Lease	Success	Success
12/23/...	0	19...	System	Renew Lease	Success	Success
12/23/...	0	19...	System	Get Instruction Set	Success	[Instruction Set = <Instru...
12/23/...	0	19...	System	Get Instruction Set	Success	[Instruction Set = <Instru...

Command/Response list Screen Shot 11